Embedded (Real-Time) operating systems

Gábor Naszály
<naszaly@mit.bme.hu>

Department of Artificial Intelligence and Systems Engineering

Terminology

Embedded operating systems

- Operating systems for **embedded systems**... ☺

Real-time operating systems

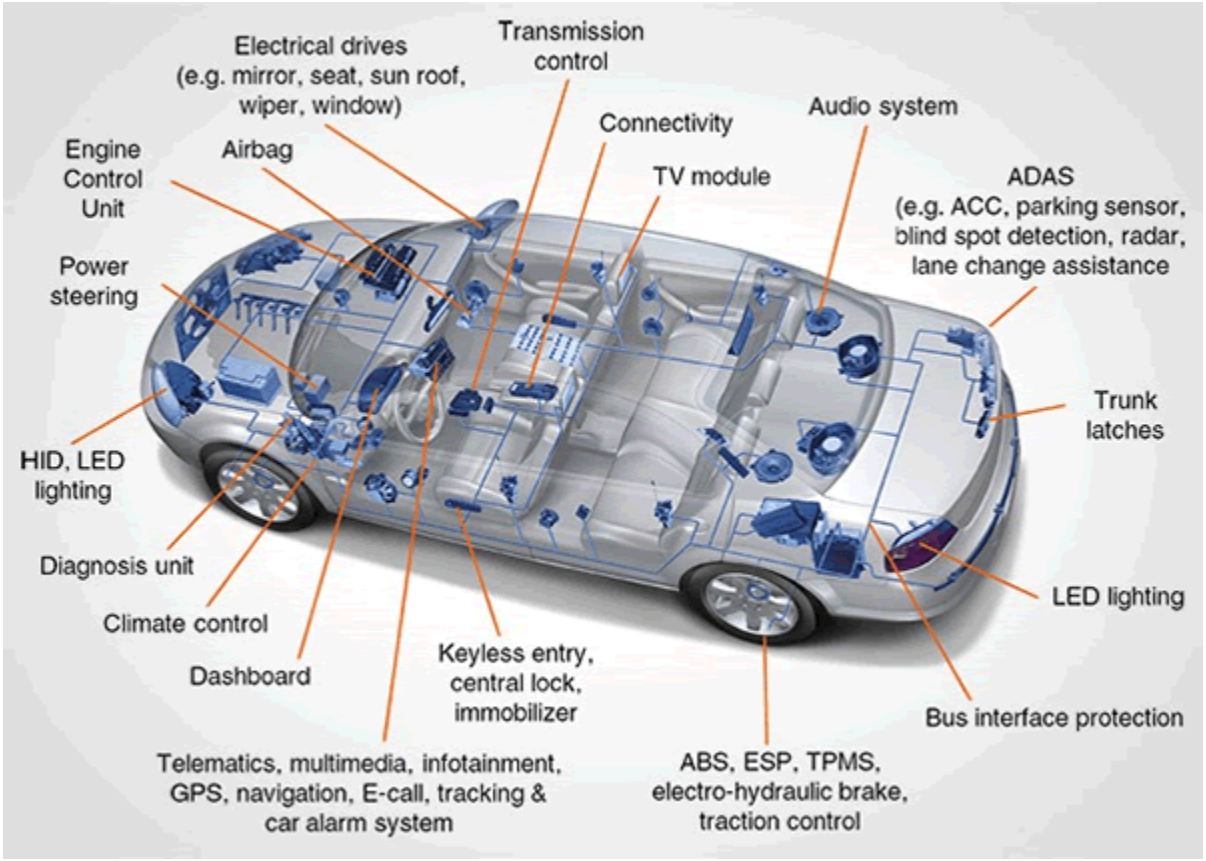
- Operating systems for **real-time systems**... ☺

Embedded systems

- Lightweight definition:
  - „All computer systems, that are not an ordinary computer.”

Embedded systems

- Areas of application – Automotive industry



Embedded systems

- Areas of application – Entertainment devices

Embedded systems

- Areas of application – Medical instruments

Embedded systems

- Areas of application – Household appliances

Embedded systems

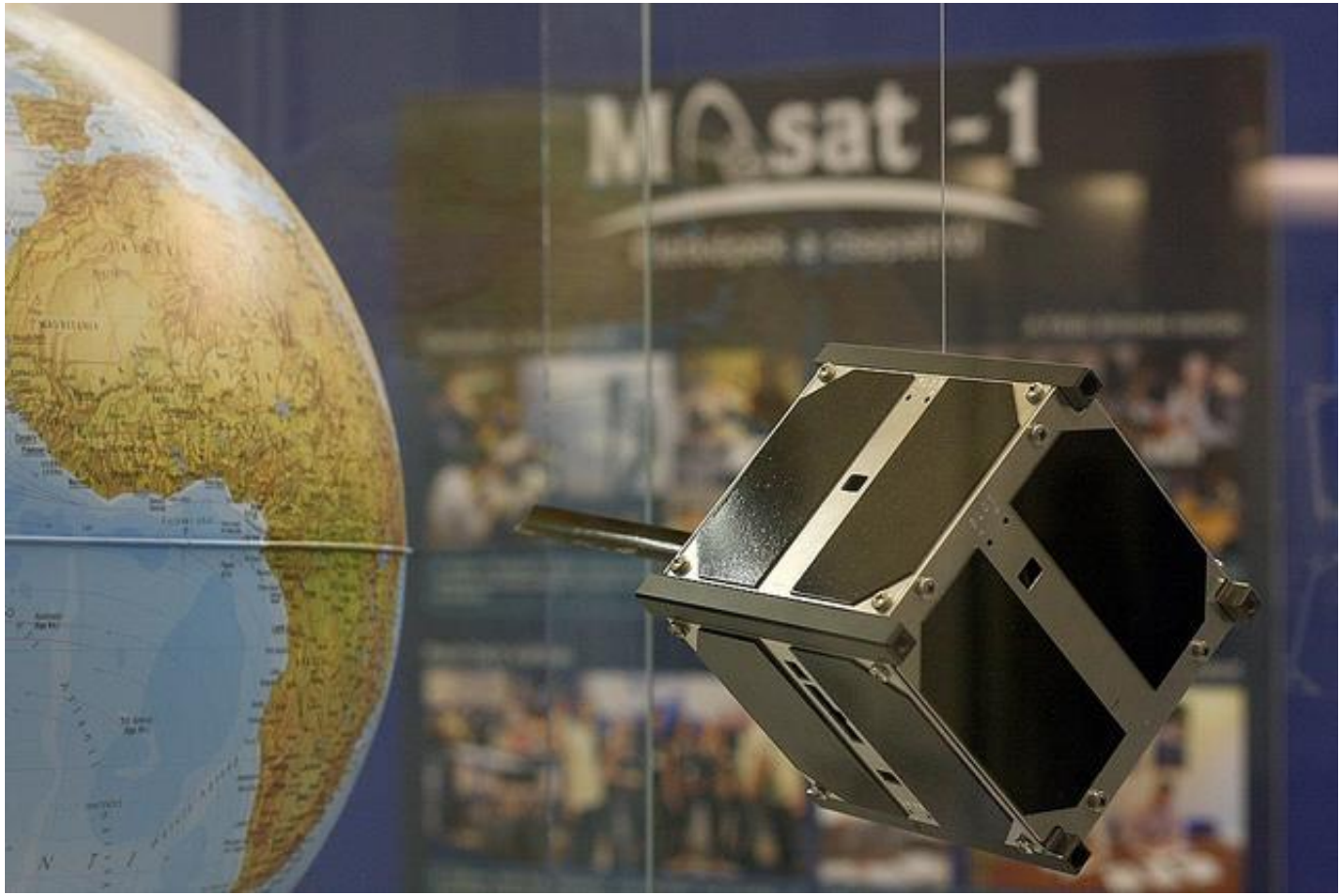
- Areas of application – Measurement instruments

Embedded systems

- Areas of application – Network equipment

Embedded systems

- Areas of application – Space industry



Embedded systems (a more accurate definition)

- Special **computer systems**, designed for **specific tasks**
- They maintain an **intense information connection towards their environment**
- Typical components:
  - Some kind of a **computing element** (like a microcontroller (MCU), or a digital signal processor (DSP), or a programmable logic (like an FPGA))
  - **Sensors** to measure some parameters of their environment
  - **Actuators** to intervene to their environment (like a valve)
  - **Communication interfaces** to communicate with other electronic components (like UART, Ethernet…)
  - **User interface for humans** (like push buttons and LEDs, or a touchscreen)

Embedded systems

- Their computing element often has a lot of various so-called **peripherals** beside the CPU:
  - Memories: program (Flash), data (SRAM)
  - Analog to Digital / Digital to analog converters (ADC, DAC)
  - Timers
  - Communication interfaces
    - Simpler ones: UART/USRT (USART), I2C, SPI
    - More complicated: USB (device, host), Ethernet (100M)
    - Field specific: CAN, LIN, FlexRay (these are automotive busses)
  - General purpose digital I/O (GPIO)
    - Simple actions (like turning on an LED, or read the state of a button)
    - To implement (by software) a special communication not supported by any other dedicated peripherals

Embedded systems

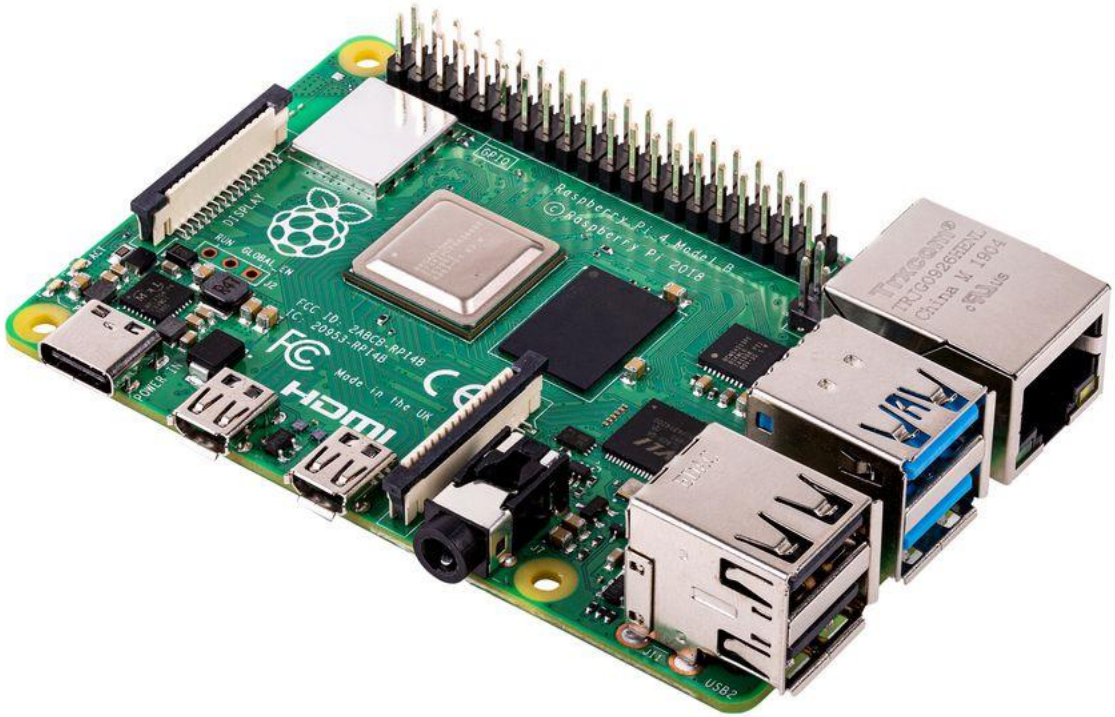
- I refer to the embedded systems we talked about so far as **classic embedded systems**
- Between classic embedded systems and ordinary personal computers (PCs) there is a transition zone:
  - **High-end embedded systems** (from the classic embedded systems' point of view)
  - **Embedded computers** (from the ordinary personal computers' point of view)

Embedded systems

- Application examples for non-classic embedded systems – Industrial PCs

Embedded systems

- Application examples for non-classic embedded systems – Card sized PCs



Embedded systems

- Application examples for non-classic embedded systems – Smartphone

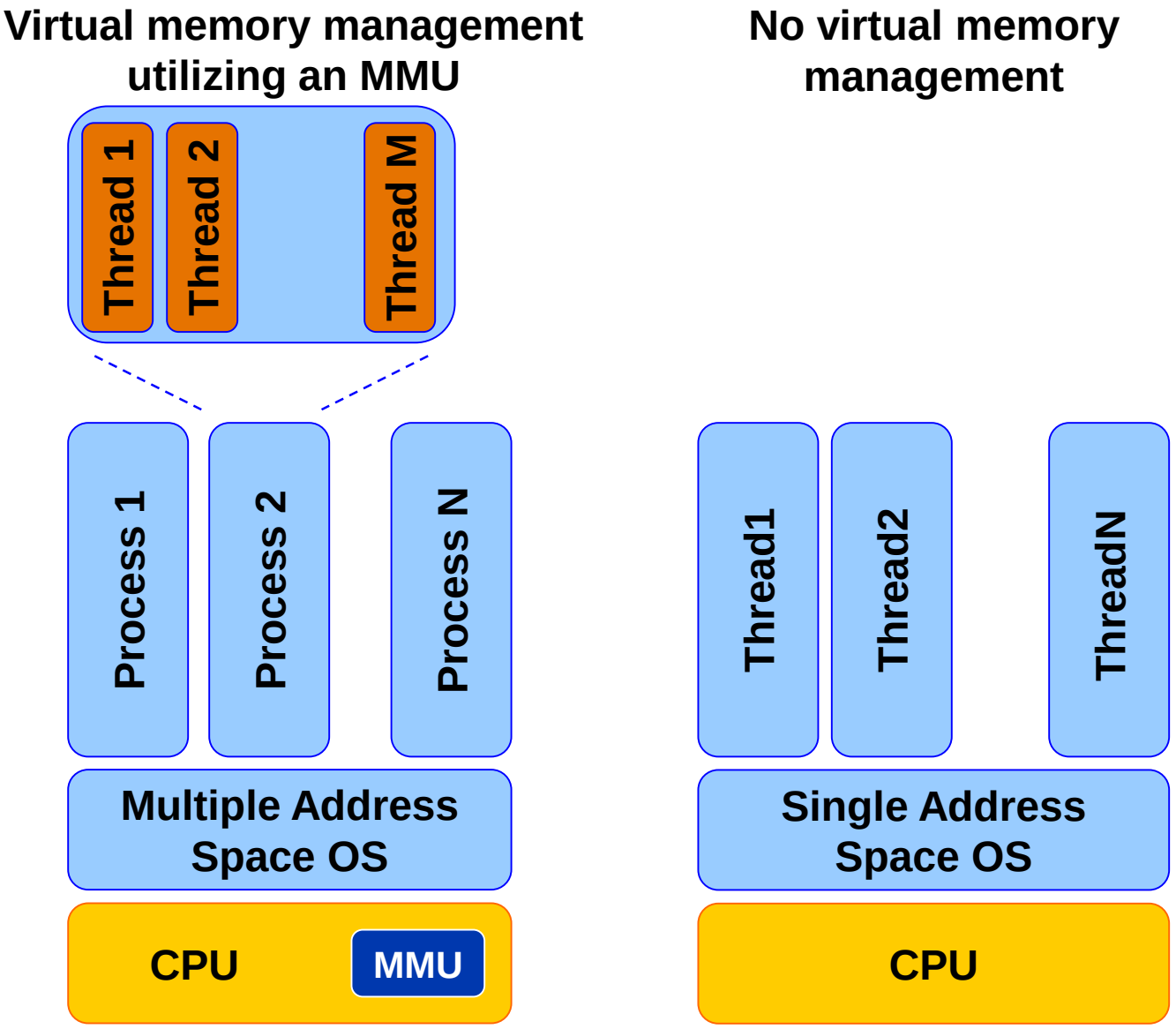
Classic embedded system ⟷ **An ordinary PC**

Dedicated task ⟷ General purpose

Embedded systems

	Classic embedded system	High-end embedded system (or a PC)
Clock frequency for the CPU	~10 – x100 MHz	~ xGHz
Protection levels in the CPU	None (or few)	Yes
MMU (Memory Management Unit – For virtual memory management)	None	Yes
Can run processes or just threads?	Just threads	Processes (and threads inside processes)
Can run a Linux like OS?	No	Yes

Embedded systems



Real-time systems

- The system is required to react to events within a well defined time period
- This time period is often small (but not necessarily)

Real-time systems

- Grouping (strict method)
  - A system is either real-time (keep deadlines) or not, there is no “in-between”

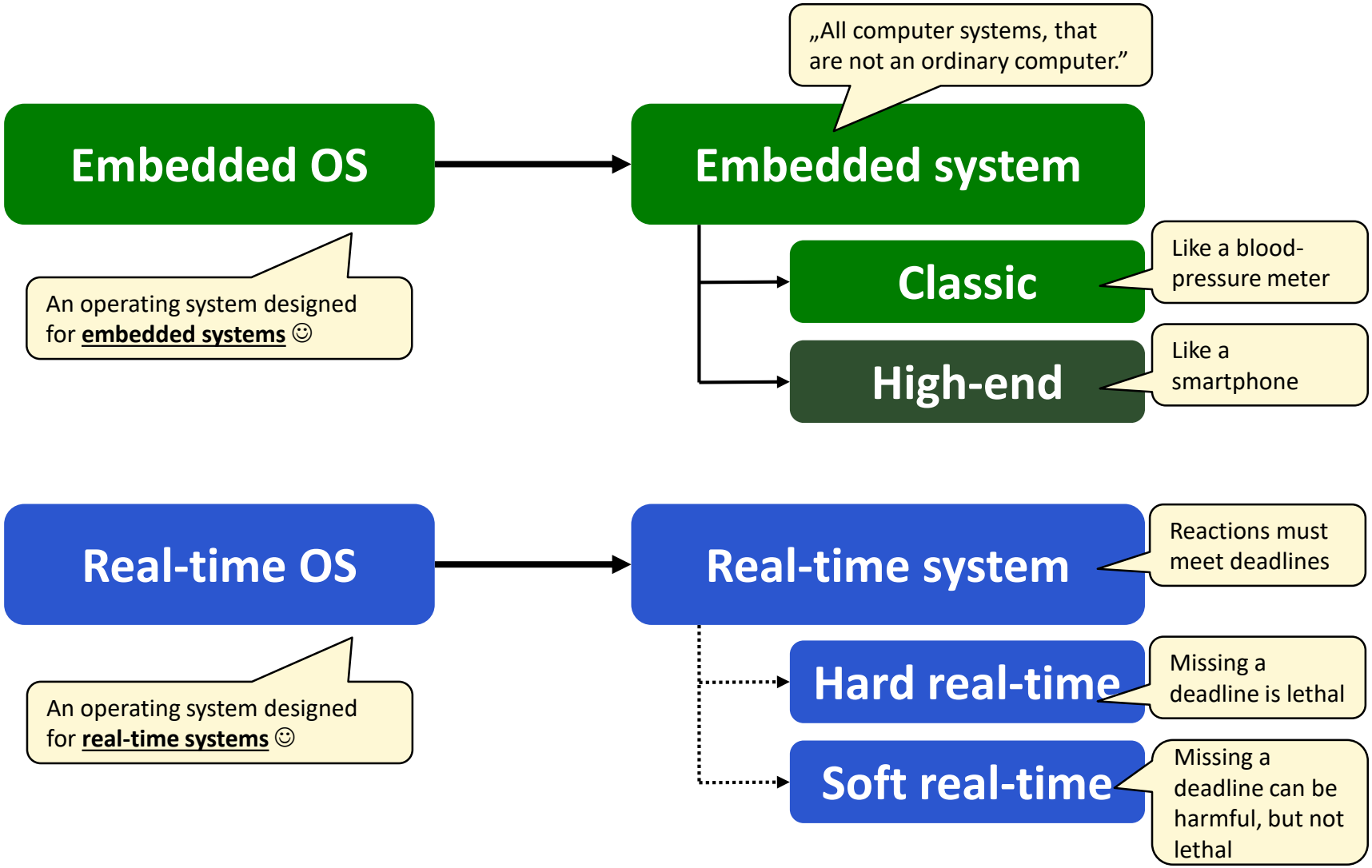
Real-time systems

- Grouping (a more relaxed method)
  - Hard real-time system
    - The system needs to be implemented in such a way, that all deadlines are met.
    - Missing a deadline can cause catastrophes (like in aviation).
  - Soft real-time system
    - Deadlines are met in a best effort manner.
    - Missing a deadline is at least inconvenient or even can be harmful (but not catastrophic). For example, an ATM gives our money after a few minutes delay.

Embedded and real-time systems

- In most of the cases, classic embedded systems are faced with real-time requirements as well.
- For this reason, the intersection of these systems is big.

Terminology

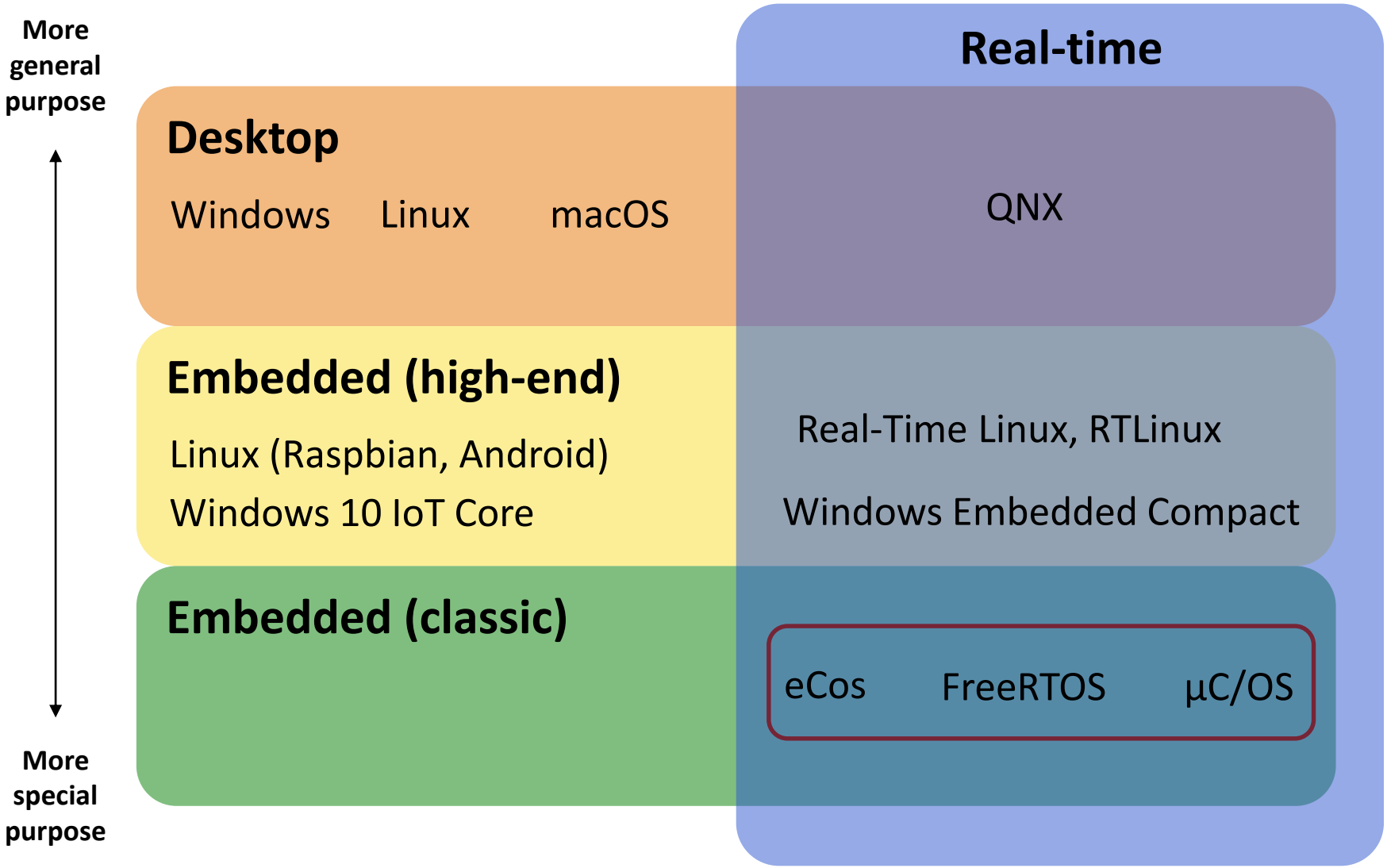


The intersection of classic embedded and real-time systems are big

Grouping operating systems

Regarding the level of embedded and real-time behavior

Grouping operating systems



The RTOS for classic embedded systems

As a software architecture

A way to RTOS (as a software architecture)

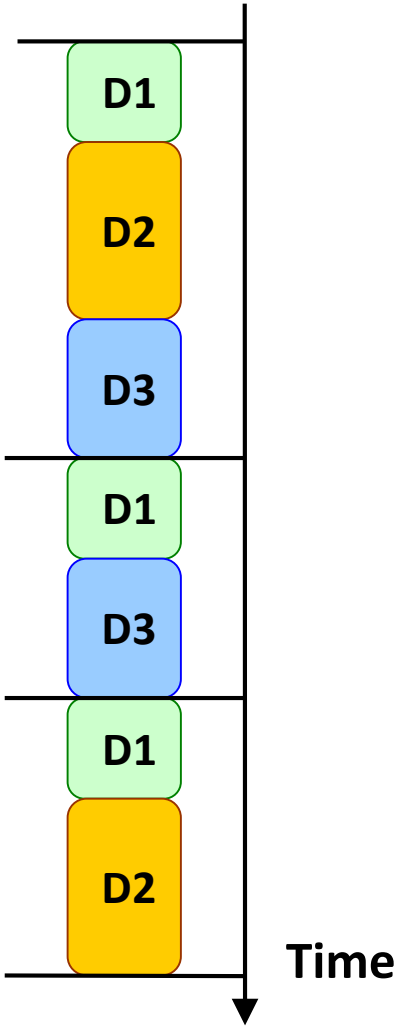
- In the beginning **assembly** (the lowest level language for an architecture) was used to implement software for embedded systems
- Today **C** is the prevailing programming language for embedded systems *(assembly is used only in some special cases, like the lowest level part of a code (closest to the CPU), or where a piece of code really needs to be efficient)*

A way to RTOS (as a software architecture)

- In the beginning, an RTOS was not used (and this is the case often even today)
- However, as embedded software became more-and-more complicated, more-and-more sophisticated software architectures were needed (as it was more-and-more difficult to achieve real-time behavior)
- We can think of the RTOS as the most sophisticated software architecture
- Let’s see some of them, starting with the simplest

Round-robin

```
void main(void) {

  while(1) {

    if ( !! Device 1 needs service ) {
       !! Handle Device 1 and its data
    }

    if ( !! Device 2 needs service ) {
       !! Handle Device 2 and its data
    }

    if ( !! Device 3 needs service ) {
       !! Handle Device 3 and its data
    }

    ...

  }

}
```



Round-robin

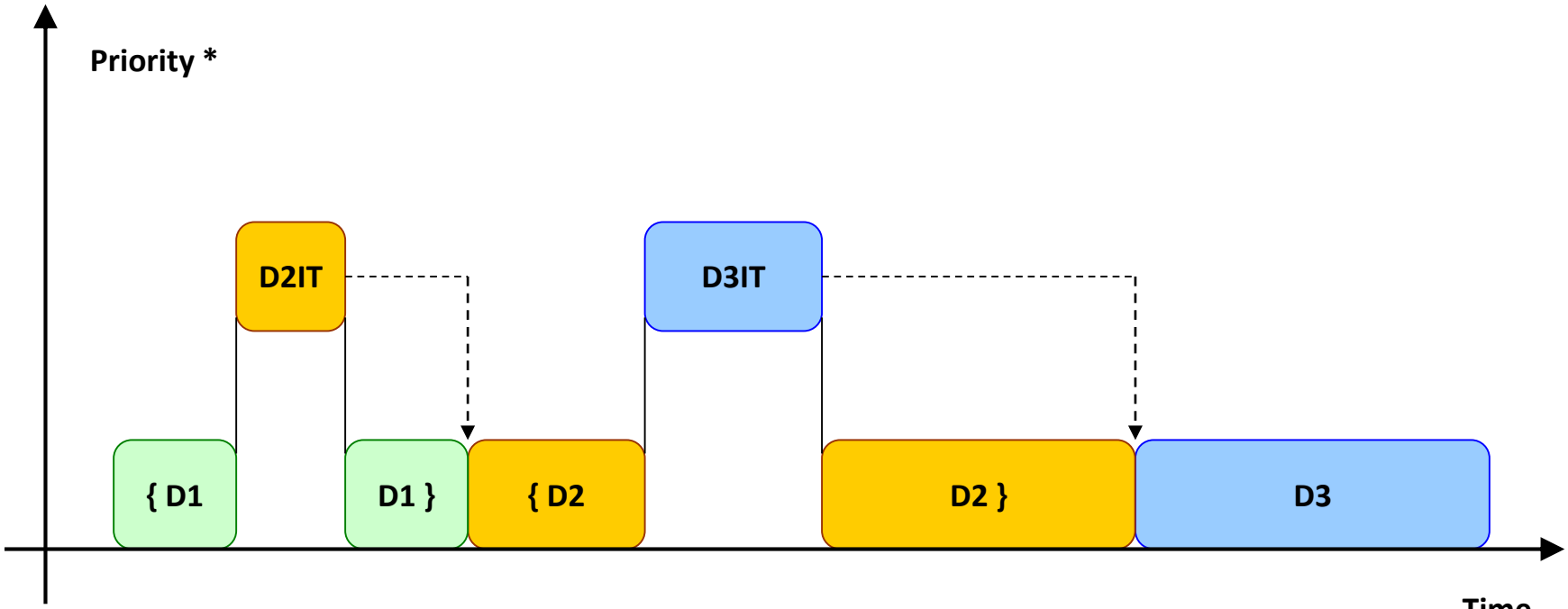
- This is the simplest
- There is no interrupts → There is no problem with shared resources
- The response time for any task varies greatly
  - Worst-case response time: the sum execution time for all tasks (increase linearly with the number of tasks)
- Tasks having more strict deadlines (we say, they have higher priority) can be polled several times
- The architecture is „fragile”: adding a new task can break it

Round-robin (with interrupts)

```
BOOL Device1_flag = FALSE;
BOOL Device2_flag = FALSE;
BOOL Device3_flag = FALSE;

void interrupt Device1_ISR (void) {
   !! Handle Device 1 time critical part
   Device1_flag = TRUE;
}

void interrupt Device2_ISR (void)  {
   !! Handle Device 2 time critical part
   Device2_flag = TRUE;
}

void interrupt Device3_ISR (void)  {
   !! Handle Device 3 time critical part
   Device3_flag = TRUE;
}
```

```
void main (void)
{
  while(1)
  {
     if ( Device1_flag ) {
       Device1_flag = FALSE;
          !! Handle Device 1 and its data
     }

     if (Device2_flag ) {
       Device2_flag = FALSE;
          !! Handle Device 2 and its data
     }

     if (Device3_flag ) {
       Device3_flag = FALSE;
          !! Handle Device 3 and its data
     }

     ...
  }
}
```

Round-robin (with interrupts)



*

- By depicting interrupts with higher priority, we refer only to the fact, that the execution of an interrupt service routine (ISR) can interrupt the execution of normal code.
- Furthermore, there are processor systems, where priorities can be assigned to interrupt requests (which means, that if there are more than one pending interrupt requests, the one with higher priority executes first). In some architectures it is even possible for a higher priority ISR to interrupt the execution of a lower priority ISR.

Round-robin (with interrupts)

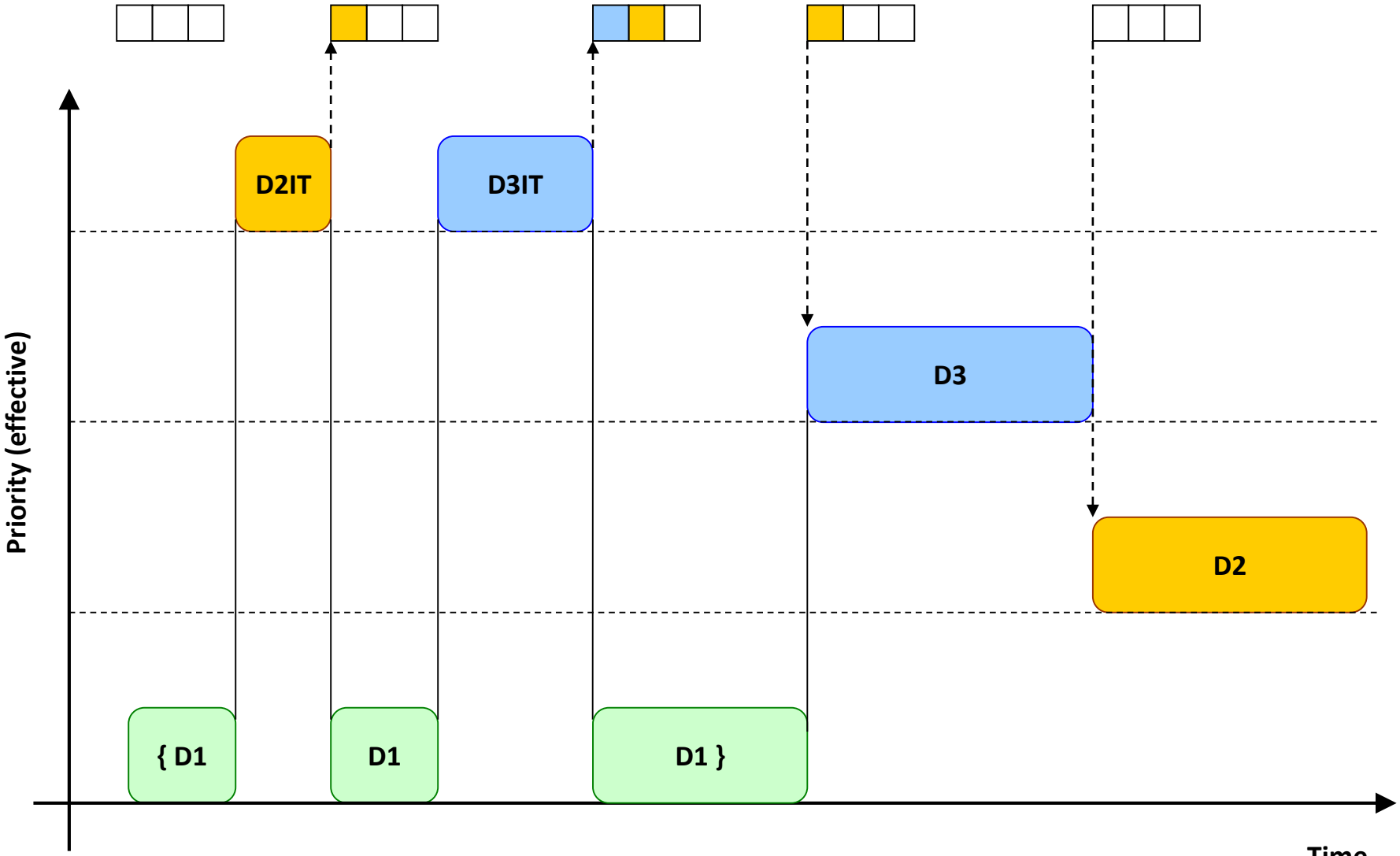
- Compared to the round-robin architecture:
  - **Time critical parts are handled better (+)**
    - We put time critical parts into ISRs, and ISRs can interrupt the execution of the main loop at any point.
    - If the HW architecture allows us to assign different priority levels to interrupts, we can refine even further our SW architecture.
  - **There can be problems with shared resources (–)**
    - A shared resource can be anything (e.g., a device or a variable) that is accessed by both an ISR and the main loop.
    - If the ISR interrupts the main loop at a point where it handles the shared resource, the resource can go into an inconsistent state.
    - This can be avoided by disabling the interrupt during the execution of the main loop where it accesses the shared resource.

Function queue scheduling

```
!! Define queue of function pointers

void interrupt Device1 (void)
{
   !! Handle Device 1 time critical part
   !! Put Device1_func to call queue
}

void interrupt Device2 (void)
{
   !! Handle Device 2 time critical part
   !! Put Device2_func to call queue
}

void interrupt Device3 (void)
{
   !! Handle Device 3 time critical part
   !! Put Device3_func to call queue
}
```

```
void main (void)
{
  while(1)
  {
     while(!! Function queue not empty)
              !! Call first from queue
  }
}

void Device1_func (void)
{   !! Handle Device 1   }

void Device2_func (void)
{   !! Handle Device 2   }

void Device3_func (void)
{   !! Handle Device 3  }
```

Function queue scheduling



Function queue scheduling

- Compared to the round-robin (with interrupts) architecture:
  - **Capable of handling priorities at task level (+):**
    - Worst-case response time (for the highest priority task) = longest task response time + IT
    - Worst-case response time does not increase linearly with the number of tasks
    - Response time is much less variable
    - A new task does not disrupt the existing schedule

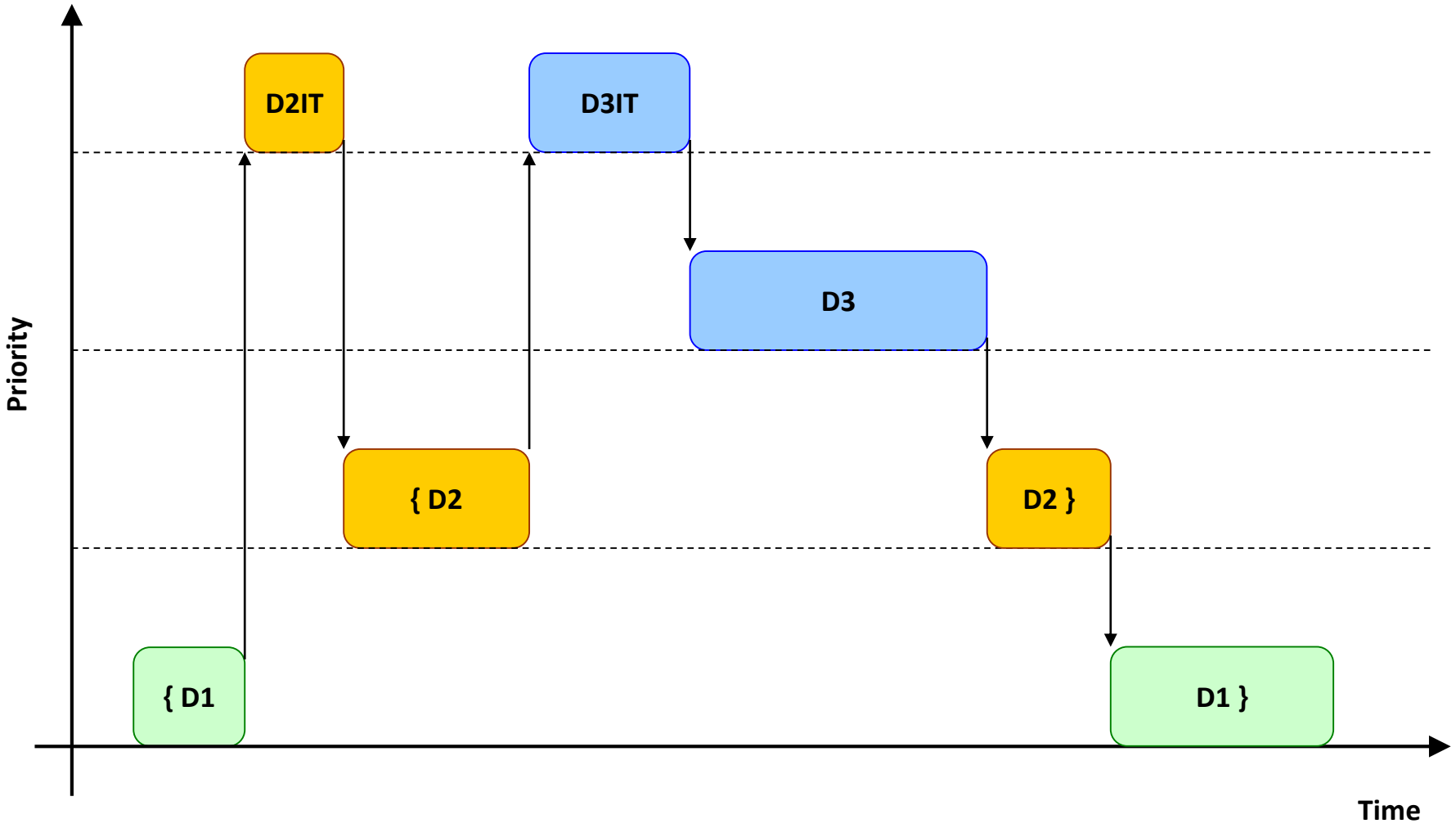
Real-time operating system (RTOS)

```
void interrupt Device1 (void) {
   !! Handle Device 1 time critical part
   !! Set signal to Device1_task
}

void interrupt Device2 (void) {
   !! Handle Device 2 time critical part
   !! Set signal to Device2_task
}

void interrupt Device3 (void) {
   !! Handle Device 3 time critical part
   !! Set signal to Device3_task
}
```

```
void main (void) {
  !! Initialize OS
  !! Start OS scheduler
}
```

```
void Device1_task (void) {
  while (1) {
    !! Wait for signal to Device1_task
    !! Handle Device 1
  }
}

void Device2_task (void) {
  while (1) {
    !! Wait for signal to Device2_task
    !! Handle Device 2
  }
}

void Device3_task (void) {
  while (1) {
    !! Wait for signal to Device3_task
    !! Handle Device 3
  }
}
```

Real-time operating system (RTOS)



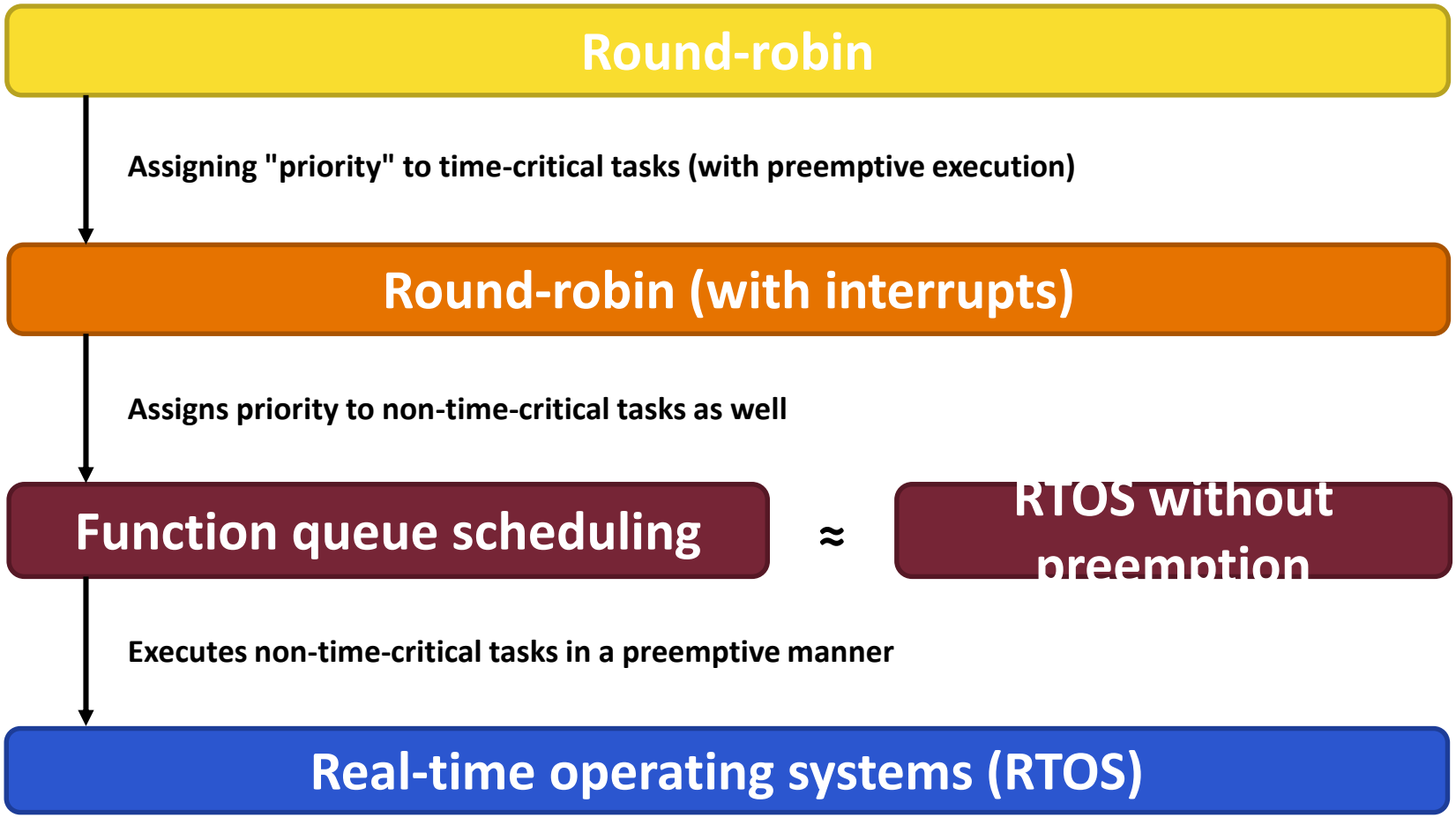
Real-time operating system (RTOS)

- Compared to the function queue scheduling architecture:
  - The scheduling is (by default) preemptive *(a higher priority task can interrupt the execution of a lower priority task)*:
    - Worst-case response time (for the highest priority task) = task switching time + IT (both small) (+)
    - Response time variance is even lower (+)
    - Shared resource problems may also between individual tasks (–)
  - Code overhead (–)

Real-time operating system (RTOS)

- Typical representatives in this category
  - FreeRTOS (originally Real Time Engineering Ltd., now Amazon Web Services)
  - µC/OS-II, µC/OS-III (originally Micriµm, now Silabs)
  - eCOS
  - Keil RTX
  - Freescale/NXP MQX
  - Express Logic ThreadX
  - TI DSP/BIOS and TI-RTOS
  - and many others...

Comparison of software architectures



RTOSs also provide various services for communication between tasks, eliminating the problem of shared resources, etc.

FreeRTOS

www.freertos.org

History

- FreeRTOS has a 20-year history:
  - It was originally developed by **Richard Barry** around 2003
  - Later, Richard's company, **Real Time Engineers Ltd.,** continued its development
  - In 2017, **Amazon Web Services** (AWS) took over the project
  - Richard continues to work on the project as a member of the AWS team that maintains it

Licensing

- MIT Open Source License:
  - Free and open source
  - Can be used in commercial applications
  - Royalty-free
  - Technical support available, but fee-based
  - No warranty
  - No legal protection
- Note:
  - Prior to version 10, a modified (simplified) version of the GNU GPLv2 license was in effect.

Members of the FreeRTOS family

Amazon Web Services:

Amazon FreeRTOS (a:FreeRTOS)

HighIntegritySystems:

OPENRTOS®

SAFERTOS®

SAFERTOS CORE

Members of the FreeRTOS family

OPENRTOS®

- WITTENSTEIN high integrity systems is behind it
- A commercially licensed version of FreeRTOS
- Not free
- In return, they provide a warranty and IP protection

Members of the FreeRTOS family

SAFERTOS®

- WITTENSTEIN high integrity systems is behind it
- Based on the FreeRTOS functional model, but not identical to it

- It has been completely reworked and built from the ground up with safety-critical principles in mind. There is no dynamic memory allocation and more thorough parameter checking for OS calls.

Members of the FreeRTOS family

Amazon FreeRTOS (a:FreeRTOS)

- Developed by Amazon Web Services
- FreeRTOS kernel + other software packages:
  - Communication (e.g., TCP/IP, MQTT)
  - Encryption (e.g. TLS)
- Goal: to make communication between embedded devices and with the cloud, as well as remote updates, easy and secure.
- MIT license

Key features of FreeRTOS

- Its source code is designed for easy portability (currently officially supported on more than 40 embedded architectures)
- Small size, simplicity, and ease of use were important considerations during its design
- The source code comes with a wealth of demo applications to make the initial steps easier

Key features of FreeRTOS

- The unit of scheduling can be:
  - Task (most common)
  - Co-routine (simpler than task, legacy)
  - Hybrid (task + co-routine)
- Scheduling of tasks:
  - Priority based
  - Preemptive (by default)
  - Round-robin time-slicing scheduling for tasks at the same priority levels (by default)

Main OS services

- Tasks
- Binary semaphores
- Mutexes
- Message queues
- Event groups /or event flags/
- Software timers
- Memory management

Less frequently used OS services

- Co-routines
- Tickless idle mode
- Counting semaphores
- Recursive mutexes
- Direct to task notifications
- Stream buffers és message buffers
- Blocking on multiple RTOS objects

Less frequently used OS services

- Hook functions
- Thread local storage
- Stack overflow detection
- Trace features
- Run time statistics
- Memory protection support

The structure of FreeRTOS

Application

FreeRTOSConfig.h

FreeRTOS: architecture independend code

list.c	*stream_buffer.c*	list.h	*stream_buffer.h*
tasks.c		task.h	*message_buffer.h*
queue.c		queue.h	semphr.h
event_groups.c		*event_groups.h*	
timers.c		*timers.h*	
croutine.c		*croutine.h*	FreeRTOS.h

FreeRTOS: architecture specific code

port.c portmacro.h

FreeRTOS: memory management

heap_(1-5).c

Software

CPU

Timer

Hardware

Configuration options

- The **`FreeRTOSConfig.h`** header contains several `#define` lines
  - There are functions that can be enabled/disabled, e.g.:
    - `#define configUSE_PREEMPTION ( 1 )`
    - `#define configUSE_MUTEXES ( 1 )`
    - `#define configIDLE_SHOULD_YIELD ( 0 )`
    - `...`
  - There are also quantifiable configuration options, e.g.:
    - `#define configTICK_RATE_HZ ( 100 )`
    - `#define configMAX_PRIORITIES ( 6 )`
    - `...`

Configuration options

- The OS code uses the configuration definitions:

```
/* A task being unblocked cannot cause an immediate
context switch if preemption is turned off. */
#if ( configUSE_PREEMPTION == 1 )
{
    /* Preemption is on, but a context switch should
    only be performed if the unblocked task has a
    priority that is equal to or higher than the
    currently executing task. */
    if( pxTCB->uxPriority >= pxCurrentTCB->uxPriority )
    {
        xSwitchRequired = pdTRUE;
    }
    else
    {
        mtCOVERAGE_TEST_MARKER();
    }
}
#endif /* configUSE_PREEMPTION */
```

Részlet a tasks.c forrás fájlból

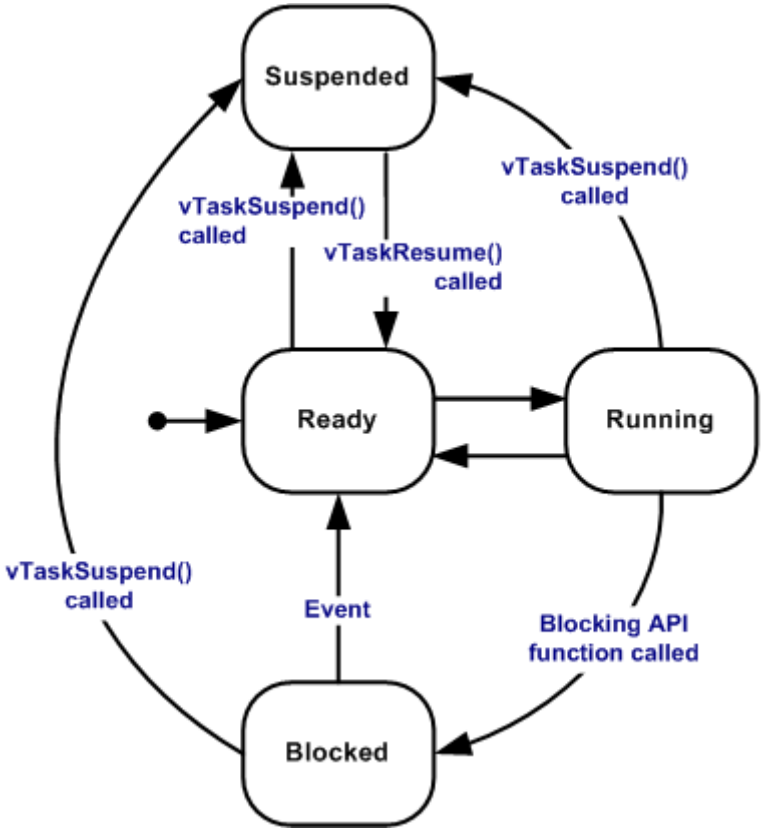
Configuration options

- The OS code uses the configuration definitions:

```
/* Setup the systick timer to generate the tick interrupts at the
 * required frequency. */
__attribute__(( weak )) void vPortSetupTimerInterrupt( void )
{
/* Calculate the constants required to configure the tick interrupt */

    /* Stop and clear the SysTick. */
    portNVIC_SYSTICK_CTRL_REG = 0UL;
    portNVIC_SYSTICK_CURRENT_VALUE_REG = 0UL;

    /* Configure SysTick to interrupt at the requested rate. */
    portNVIC_SYSTICK_LOAD_REG =
        ( configSYSTICK_CLOCK_HZ / configTICK_RATE_HZ ) - 1UL;
    portNVIC_SYSTICK_CTRL_REG =
        ( portNVIC_SYSTICK_CLK_BIT |
          portNVIC_SYSTICK_INT_BIT |
          portNVIC_SYSTICK_ENABLE_BIT );
}
```

Részlet a /portable/GCC/ARM_CM3/port.c forrás fájlból (az olvashatóság kedvéért picit átszerkesztve)

Task states


Suspended
vTaskSuspend()
called
vTaskResume()
called
vTaskSuspend()
called
Ready
Running
vTaskSuspend()
called
Event
Blocking API
function called
Blocked

Typical application

- `main()`:
  - Initialization
  - Task creation → `xTaskCreate()`
  - Starting the scheduler → `vTaskStartScheduler()`
    - This function does not return to main(). The OS always schedules one of the tasks. If none of our own tasks are ready to run, the built-in idle task will run.

Typical application

- Tasks
  - Implemented in functions
  - Two types of task structure are possible:
    - Single-shot:
      - The task runs once
      - During its execution, it can trigger events that cause other tasks to run
      - Then deletes itself at the end of its run → `vTaskDelete()`
    - Infinite loop:
      - There may be an optional initialization phase at the beginning
      - Followed by an infinite loop
      - In which you must place an OS call that puts it into a waiting state (so that it does not starve lower priority tasks)

Simple FreeRTOS example application

Demo

The implemented function

- Two tasks: one high priority and one low priority
- Both are implemented as infinite loops
- Within their loop:
  - They print a short text message to the standard output ("Hi" or "Lo")
  - Then they put themselves into a waiting state (the high priority for one second, the low priority for half a second)

Application code

```c
int main(void) {
   [...] // Init stdio: use UART0

   xTaskCreate(
      prvTaskHi,
      "Hi",
      mainTASK_HI_STACK_SIZE,
      NULL,
      mainTASK_HI_PRIORITY,
      NULL);

   xTaskCreate(
      prvTaskLo,
      "Lo",
      mainTASK_LO_STACK_SIZE,
      NULL,
      mainTASK_LO_PRIORITY,
      NULL);

   vTaskStartScheduler();
   return 0; // Never reached
}
```

```c
static void prvTaskHi(void *pvParam) {
   while (1) {
      printf("Hi\n");
      vTaskDelay(configTICK_RATE_HZ);
   }
}

static void prvTaskLo(void *pvParam) {
   while (1) {
      printf("Lo\n");
      vTaskDelay(configTICK_RATE_HZ / 2);
   }
}
```

prvTaskHi, prvTaskLo: Function pointers to the functions that host the task code.

mainTASK_HI_STACK_SIZE, mainTASK_LO_STACK_SIZE: Constants specifying the size of the task stacks. It is important that they are sufficiently large. In embedded systems, **printf()** can be a relatively large stack consumer...

mainTASK_HI_PRIORITY, mainTASK_LO_PRIORITY: Task priorities: low is 1, high is 2 (the idle task has a priority of 0).

Hardware used

- The device used is the Silicon Labs EFM32 Giant Gecko Starter Kit (STK3700):

Debugger circuitry (connection to computer):

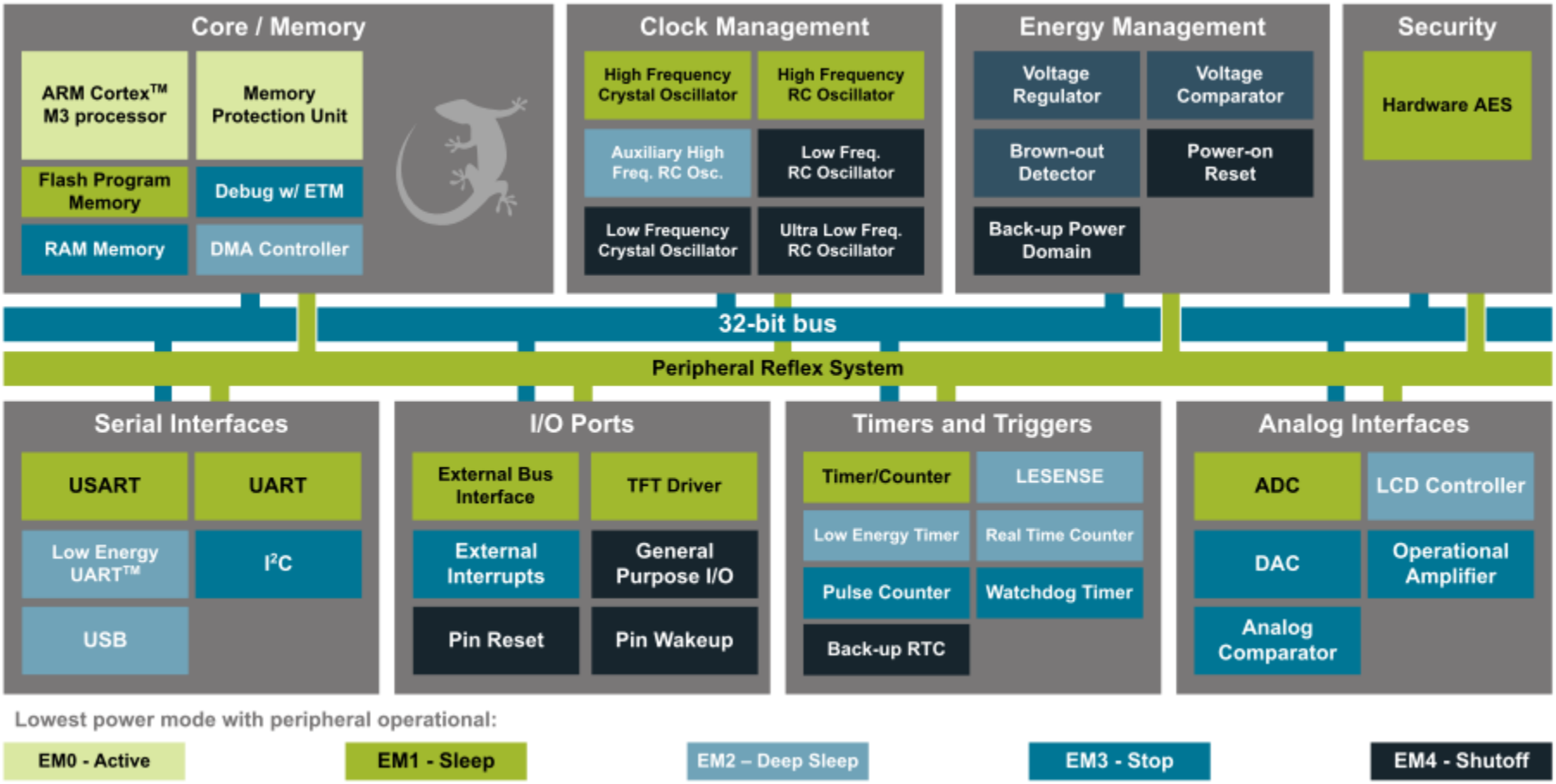
- Program download to microcontroller, debugging
- Relays the communication of one of the microcontroller's UART peripherals over the USB (virtual serial port on the computer)
- One possible power source for the microcontroller. If powered from here, the current consumption can be observed.

EFM32GG990F1024 microcontroller:

- 32-bit core (ARM Cortex-M3)
- 1 MiB program memory (flash)
- 128 KiB data memory (SRAM)

Hardware used

- Simplified block diagram of the microcontroller located on the development board:



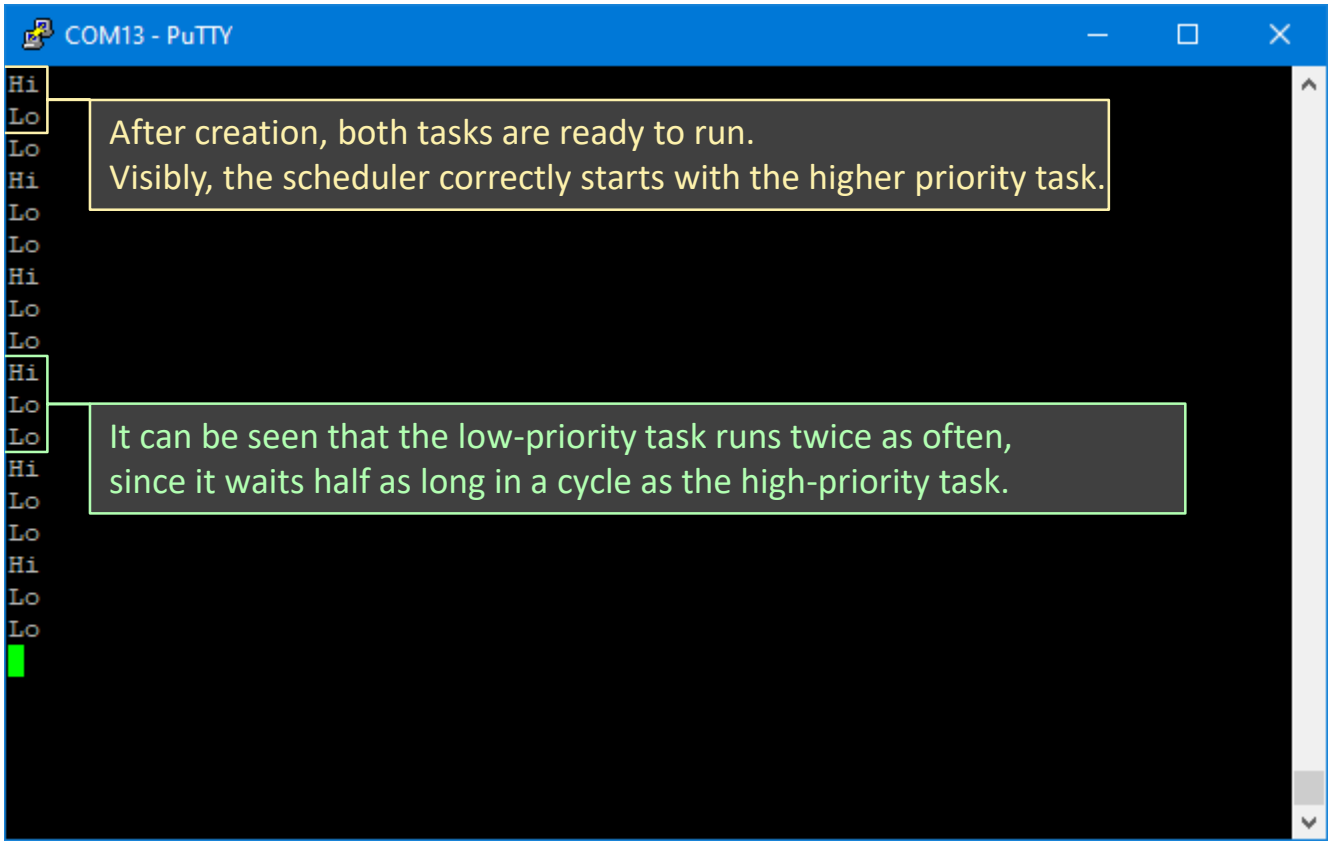
Hardware used

- The example application uses the following from the microcontroller:
  - ARM Cortex-M3 (CPU) → program execution
  - Flash Memory → program memory for the code
  - RAM Memory → data memory for variables
  - UART0 → simple serial communication for C stdio (`printf()` writes here)
  - System Timer → for tracking the passage of time (handled by FreeRTOS in interrupt mode; this unit is not directly visible in the previous block diagram; it is the built-in timer of the ARM Cortex-M3)

Hardware used

- The path of printf() to the computer:
  - The embedded application output uses one of the microcontroller's UART peripherals to send characters.
  - This arrives to the debugger circuitry, which transmits it to the computer via a USB connection.
  - A driver creates a virtual serial port on the computer (called COM<#> under Windows).
  - Terminal programs (such as PuTTY) can connect to COM ports. This allows the characters received by the PC to be displayed in the application window, and the characters typed to be sent (our embedded example application ignores the received characters).

Output generated by the application



Application tracing

- Many RTOSs (including FreeRTOS) provide the ability to monitor their execution → trace services
- In practice, this means that trace macros are located at several points in the OS code, e.g.:
  - When a task leaves the *Running* state
  - When a task enters the *Running* state
  - When a clock tick occurs (System Timer interrupt)
  - When a task starts waiting for a semaphore
  - ...

Application tracing

- These macros are empty by default:

```
#ifndef traceTASK_SWITCHED_IN
    /* Called after a task has been selected to run.
       pxCurrentTCB holds a pointer to the task control block
       of the selected task. */

    #define traceTASK_SWITCHED_IN()

#endif
```

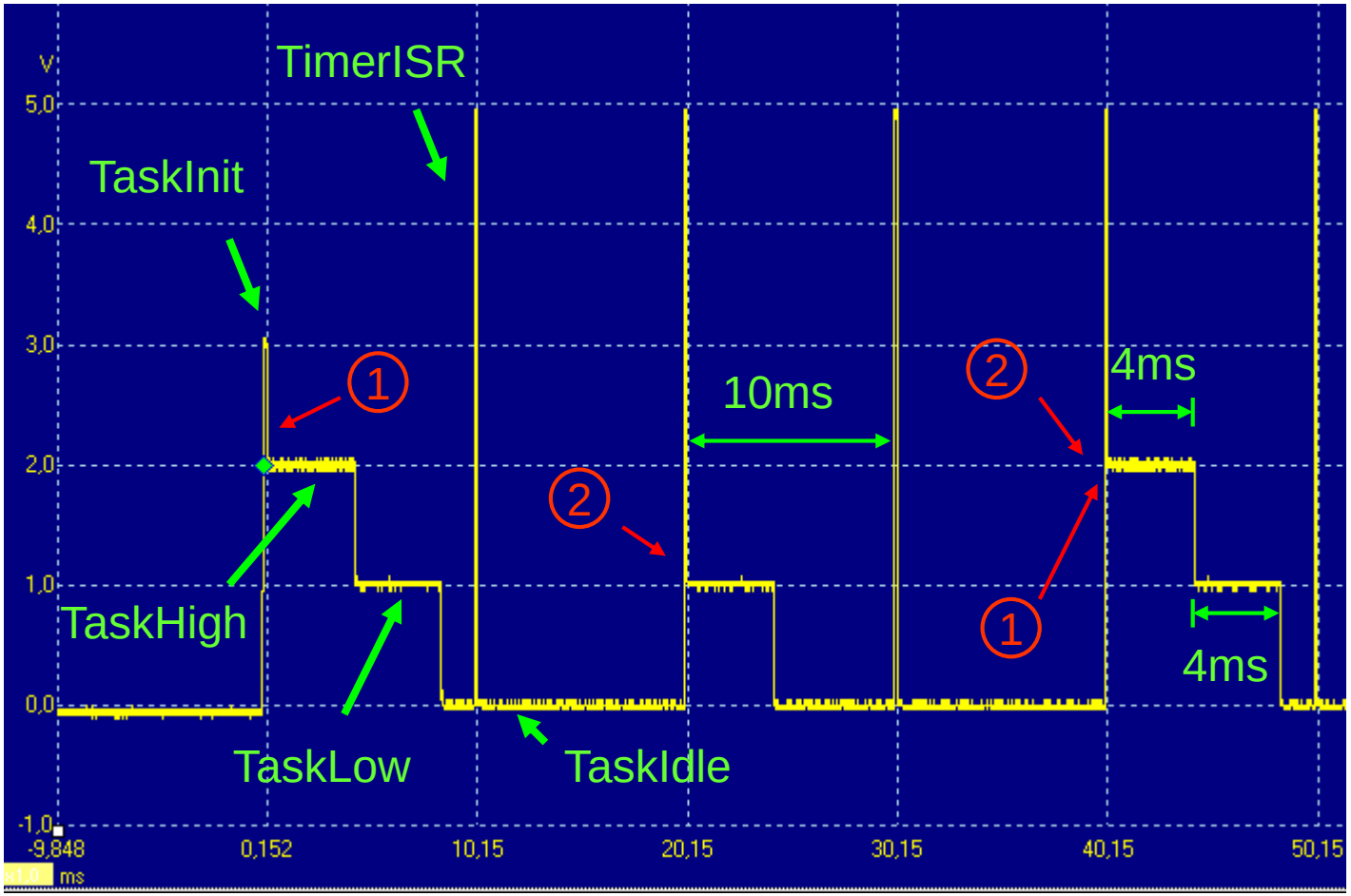
Excerpt from the FreeRTOS.h header file

- However, it is clear that they can be overridden

Application tracing

- In one of the simplest cases, trace macros can be overridden so that the application always sets an analog output to a voltage level proportional to the priority of the currently running task.
- In addition, interrupts (such as clock ticks in most RTOSs) can also be assigned a voltage level higher than that assigned to tasks.
- The output can be monitored with an oscilloscope to track the application's execution.

Application tracing



This diagram was created earlier using a different (but similar) example application under a different (but similar) operating system (µC/OS-II). The application differs from the current one in that it also includes a third, single-shot task (TaskInit). This task has the highest priority and creates the other two tasks, then deletes itself.

1: Moments, where multiple tasks are ready to run, and the OS correctly schedules the one with the highest priority

2: Moment, where the timer interrupt doesn't return to the originally interrupted task, rather triggers a task switch

Application tracing

- There are also more sophisticated solutions than simply adjusting the voltage of an analog output pin.
- Trace macros can also be implemented to log individual events (and sometimes also to collect other useful additional information about them).
- This information can then be sent to a computer via some communication channel, where it can be analyzed much more efficiently using an analyzer application.

Application tracing

- One possible tool for this is SEGGER SystemView

Code running on the embedded device:

- Must be compiled together with our own embedded application
- Minimal overhead
- Its task is to collect information about events and then store it in a buffer in RAM
- Makes the contents of the buffer available to a computer via a debug channel

Application running on the computer:

- Its task is to display the observed events

Application tracing

- Example of a FreeRTOS trace macro implemented by SystemView:

```
#define traceTASK_SWITCHED_IN()                                          \
    if(prvGetTCBFromHandle(NULL) == xIdleTaskHandle) {                   \
        SEGGER_SYSVIEW_OnIdle();                                         \
    } else {                                                             \
        SEGGER_SYSVIEW_OnTaskStartExec((U32)pxCurrentTCB);               \
    }
```

Excerpt from SEGGER_SYSVIEW_FreeRTOS.h (reformatted)

Function provided by SystemView

Macro provided by FreeRTOS, variable

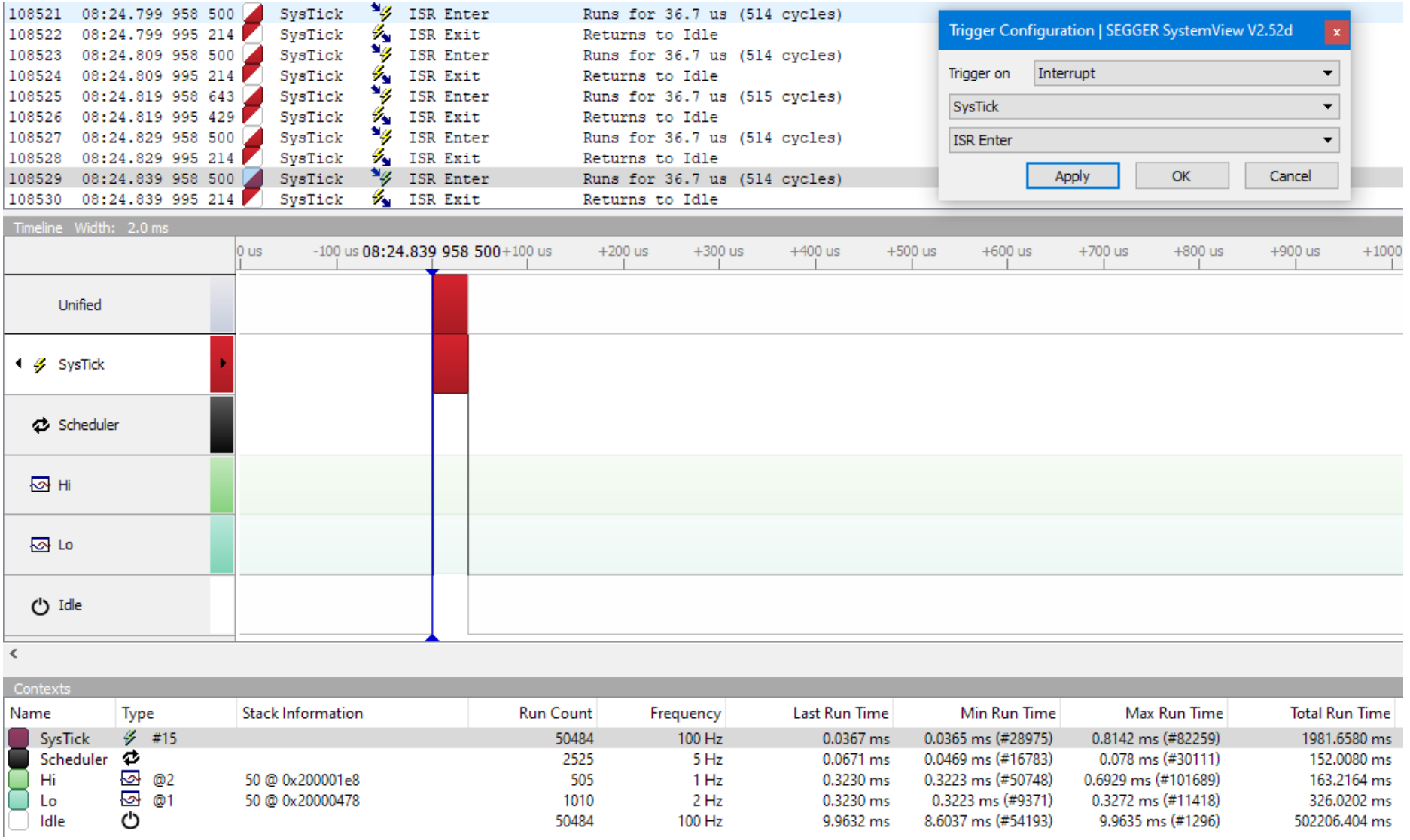
Application tracing

- Let's trace the execution of our example application using SEGGER SystemView!
- The following slides show the different phases of the execution.

Application tracing

- Our application spends most of its time in the *Idle* thread, as our own threads are only active for a short time in every 1 or 0.5 seconds.
- The *Idle* thread is interrupted periodically by the interrupt service routine for the System Timer (called the SysTick ISR).
- It is implemented by the OS. This way the OS can track the passage of time and re-schedule if needed.
- In our case, there will usually be no re-scheduling, the SysTick ISR returns to the *Idle* thread.

Application tracing

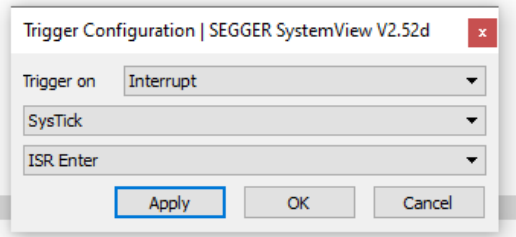


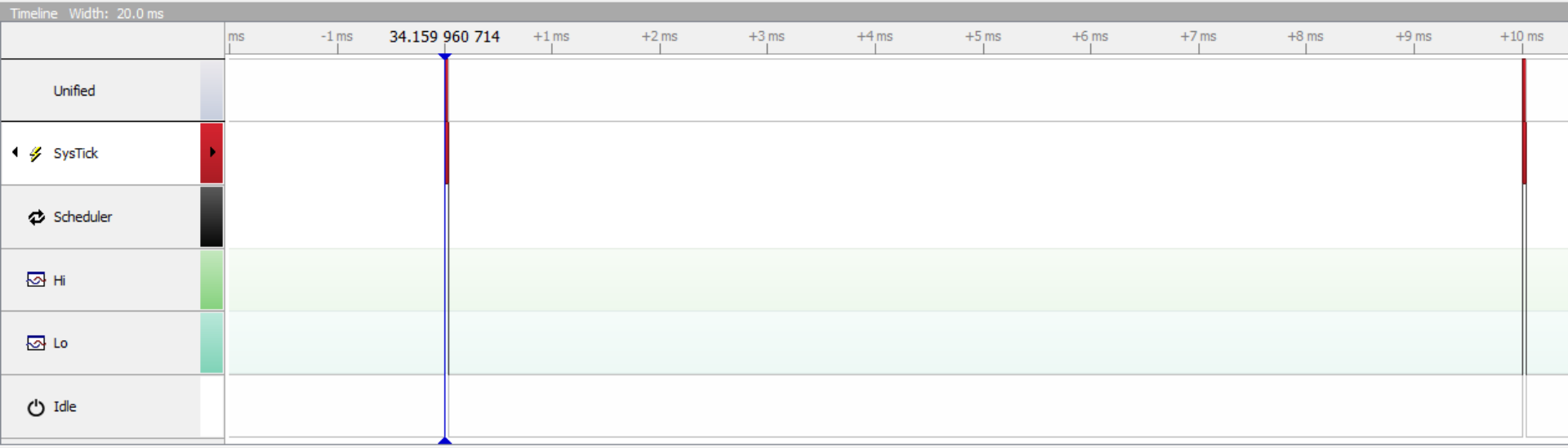
Application tracing

- Looking at a slightly longer time slice, we can see that the system is currently configured for periodic SysTick interrupts with 10 ms period.

Application tracing

7352	34.119 960 643	SysTick	ISR Enter	Runs for 36.7 us (514 cycles)
7353	34.119 997 357	SysTick	ISR Exit	Returns to Idle
7354	34.129 960 643	SysTick	ISR Enter	Runs for 36.7 us (514 cycles)
7355	34.129 997 357	SysTick	ISR Exit	Returns to Idle
7356	34.139 960 643	SysTick	ISR Enter	Runs for 36.7 us (514 cycles)
7357	34.139 997 357	SysTick	ISR Exit	Returns to Idle
7358	34.149 960 643	SysTick	ISR Enter	Runs for 36.7 us (514 cycles)
7359	34.149 997 357	SysTick	ISR Exit	Returns to Idle
7360	34.159 960 714	SysTick	ISR Enter	Runs for 36.7 us (514 cycles)
7361	34.159 997 429	SysTick	ISR Exit	Returns to Idle
7362	34.169 960 643	SysTick	ISR Enter	Runs for 36.7 us (514 cycles)





Contexts

Name	Type	Stack Information	Run Count	Frequency	Last Run Time	Min Run Time	Max Run Time	Total Run Time	Run Time/s	
SysTick	#15		3416	100 Hz	0.0367 ms	0.0366 ms (#458)	0.8123 ms (#611)	135.1815 ms	3.8333 ms	0.38%
Scheduler			172	5 Hz	0.0670 ms	0.0469 ms (#3358)	0.0738 ms (#1641)	10.296 ms	0.3007 ms	0.03%
Hi	@2	50 @ 0x200001e8	34	1 Hz	0.3230 ms	0.3224 ms (#1858)	0.3271 ms (#5294)	10.6643 ms	0.3230 ms	0.03%
Lo	@1	50 @ 0x20000478	69	2 Hz	0.3230 ms	0.3223 ms (#6904)	0.3273 ms (#6262)	21.9715 ms	0.6461 ms	0.06%
Idle			3416	100 Hz	9.9633 ms	9.0206 ms (#5299)	9.9635 ms (#4083)	33971.1935 ms	994.8967 ms	99.49%

Application tracing

- A slightly more interesting case is when one of the tasks wakes up.
- For simplicity's sake, let's first look at a time when only the low-priority thread wakes up.
- In the SysTick ISR, the OS now puts the low-priority thread into the *Ready* state (indicated by the gray bar; however, it is not running yet).
- After the ISR, the scheduler (black) will run, and switches from the *Idle* thread to the low-priority thread.

Application tracing

- After the low-priority thread has written its message to the standard output (*printf()*), it starts to wait again for half a second (*vTaskDelay()*, represented by the small vertical green line).
- As a result of this OS call, the currently running thread enters the *Blocked* state. Therefore, the OS does not immediately return from this call to the low-priority thread. Instead, the OS executes its scheduler. The scheduler figures out, that the highest priority thread among the ones ready to run is now the *Idle* thread. So a switch is made.

Application tracing

65896	05:06.519 958 500	SysTick	ISR Enter	Runs for 91.7 us (1 285 cycles)
65897	05:06.520 006 857	SysTick	Task Ready	Lo, runs after 90.4 us (1 266 cycles)
65898	05:06.520 050 286	SysTick	ISR Exit	Returns to Scheduler
65899	05:06.520 097 286	Lo	Task Run	Runs for 323.0 us (4 523 cycles)
65900	05:06.520 372 214	Lo	vTaskDelay	xTicksToDelay=50
65901	05:06.520 420 357	Lo	Task Block	Lo, Wait for timeout
65902	05:06.520 487 500	Scheduler	System Idle	Idle for 9.471 ms (132 594 cycles)
65903	05:06.529 958 500	SysTick	ISR Enter	Runs for 36.7 us (514 cycles)
65904	05:06.529 995 214	SysTick	ISR Exit	Returns to Idle
65905	05:06.539 958 500	SysTick	ISR Enter	Runs for 36.7 us (514 cycles)
65906	05:06.539 995 214	SysTick	ISR Exit	Returns to Idle

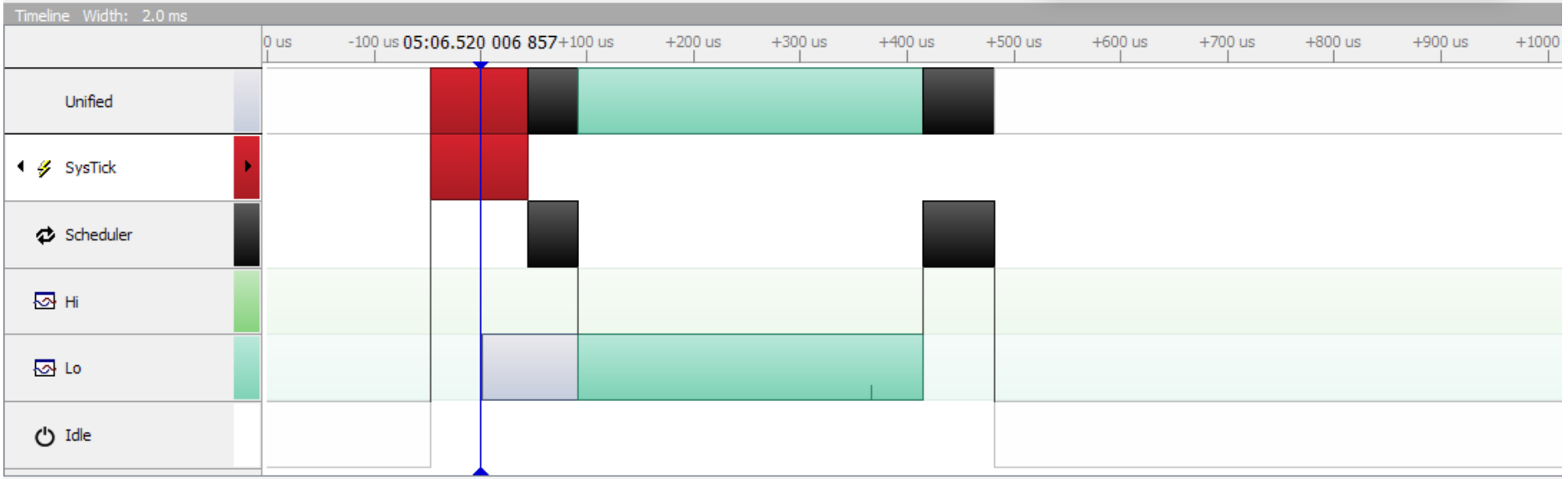
Trigger Configuration | SEGGER SystemView V2.52d

Trigger on: Task

Lo

Task Ready

Apply | OK | Cancel



Contexts

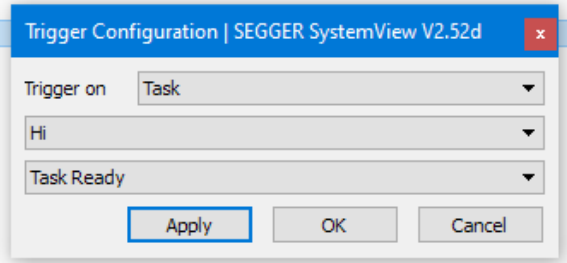
Name	Type	Stack Information	Run Count	Frequency	Last Run Time	Min Run Time	Max Run Time	Total Run Time
SysTick	#15		30657	100 Hz	0.0367 ms	0.0365 ms (#28975)	0.8122 ms (#56368)	1203.0479 ms
Scheduler			1535	5 Hz	0.0671 ms	0.0469 ms (#16783)	0.078 ms (#30111)	92.3841 ms
Hi	@2	50 @ 0x200001e8	307	1 Hz	0.323 ms	0.3223 ms (#50748)	0.3272 ms (#33983)	98.867 ms
Lo	@1	50 @ 0x20000478	614	2 Hz	0.3230 ms	0.3223 ms (#9371)	0.3272 ms (#11418)	198.063 ms
Idle			30657	100 Hz	9.9632 ms	8.6037 ms (#54193)	9.9635 ms (#1296)	304966.9447 ms

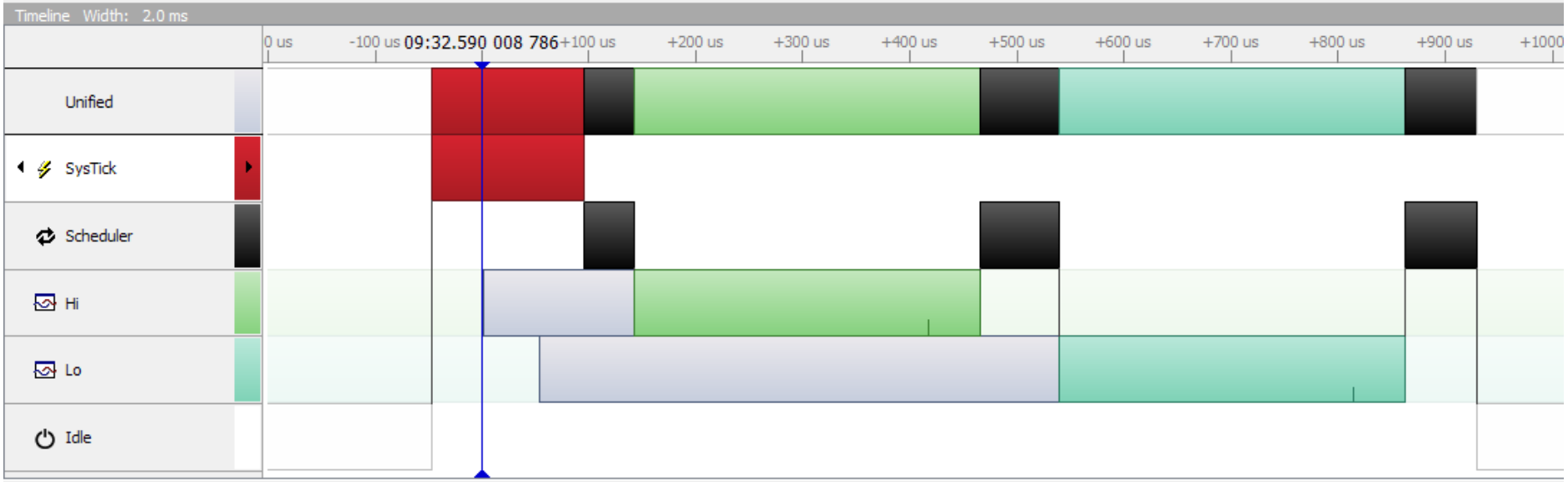
Application tracing

- The next slide shows the most complex run sequence of our simple example application. This is when both of our own threads wake up at the same time.
- You can see that the ISR now puts both of our threads into the *Ready* state. After exiting the ISR, the scheduler runs, which correctly schedules the high-priority thread first.
- This is followed later by the low-priority thread, and then the *Idle* thread.

Application tracing

123082	09:32.589 960 429	SysTick	ISR Enter	Runs for 142.7 us (1 999 cycles)
123083	09:32.590 008 786	SysTick	Task Ready	Hi, runs after 141.4 us (1 980 cycles)
123084	09:32.590 061 214	SysTick	Task Ready	Lo, runs after 485.6 us (6 799 cycles)
123085	09:32.590 103 214	SysTick	ISR Exit	Returns to Scheduler
123086	09:32.590 150 214	Hi	Task Run	Runs for 323.0 us (4 523 cycles)
123087	09:32.590 425 143	Hi	vTaskDelay	xTicksToDelay=100
123088	09:32.590 473 286	Hi	Task Block	Hi, Wait for timeout
123089	09:32.590 546 857	Lo	Task Run	Runs for 323.0 us (4 523 cycles)
123090	09:32.590 821 786	Lo	vTaskDelay	xTicksToDelay=50
123091	09:32.590 869 929	Lo	Task Block	Lo, Wait for timeout
123092	09:32.590 936 143	Scheduler	System Idle	Idle for 9.0242 ms (126 340 cycles)





Contexts

Name	Type	Stack Information	Run Count	Frequency	Last Run Time	Min Run Time	Max Run Time	Total Run Time
SysTick	#15		57263	100 Hz	0.0367 ms	0.0365 ms (#34248)	0.8125 ms (#122470)	2251.7629 ms
Scheduler			2865	5 Hz	0.0662 ms	0.0469 ms (#25931)	0.0779 ms (#27868)	172.4773 ms
Hi	@2	50 @ 0x200001e8	573	1 Hz	0.3230 ms	0.3223 ms (#14324)	0.3274 ms (#108899)	184.8201 ms
Lo	@1	50 @ 0x20000478	1146	2 Hz	0.3230 ms	0.3223 ms (#142)	0.3272 ms (#12825)	369.9553 ms
Idle			57263	100 Hz	9.9632 ms	8.6045 ms (#13473)	9.9636 ms (#112961)	569640.2910 ms

Application tracing

- Finally, to illustrate the timing relationships, let's look at how the runtime of the previous case relates to the frequency of SysTick ISRs
- The runtime of our tasks is mostly determined by the UART communication. The configured speed is 115,200 bps (let's consider it 100,000 for the sake of simplicity). The 8-bit characters are framed by 2 auxiliary bits during transmission (*Start* and *Stop*), so 10 bits are required to transmit one character, meaning the speed is 10,000 char/s. Simplified by a thousand, this is: 10 char/ms

Application tracing

- Both tasks send a two-letter word ("Hi" or "Lo") plus the "new line" character (\n), to which the API automatically inserts the other line ending character ("carriage return," \r).
- So, if both tasks wake up at the same time, they will send a total of 8 characters. At a speed of 10 char/ms, this takes a little less than 1 ms.
- The OS calls (*vTaskDelay()*), the SysTick ISR, and the scheduler runtime are shorter than this, but not zero.

Application tracing